When Does the Simple Baseline Win? Operating Regimes for Fallback and Estimation in Two-Tier Video Analytics

Amit Dua
Yushu Excellence Technologies Pvt. Ltd.
mail.amitduabits@gmail.com

2026

Abstract

Two design decisions in a deployed video-analytics pipeline look like places to add machinery, and in our deployment neither of them was. When the learned engine fails, should the pipeline fall back to a classical secondary or simply try the learned engine again? And once the output stream carries records from both paths, does stratifying a label-free accuracy estimator by that path beat an unstratified baseline? This paper answers both as *it depends, and here is the number the decision turns on* rather than as a proposal, and in both cases the simple option wins across most of the operating range.

Fallback. Retrying beats the two-tier pipeline on accuracy over all frames at every injected failure rate we tested, by 0.003 to 0.049 — but it returns nothing at all on 0.02% to 9.7% of frames, where the two-tier pipeline always returns something. Scoring a gap as m correct records converts this into a single break-even quantity, $m^\star = (a_r - a_2)/(1 - y_r)$. Measured, $m^\star$ falls from 14.5–71.5 at a 5% failure rate to 0.11–0.45 at 50%. The rule that follows is sharp: retry unless a gap in the stream costs you more than $m^\star$ correct records, and note that $m^\star$ drops below 1 once the primary is failing more than about a third of the time.

Estimation. Provenance stratification does not beat an unstratified prior at estimating whole-batch accuracy. **It does not.** Across three engine pairs, provenance stratification and a global calibration prior are indistinguishable on whole-batch accuracy (absolute error 0.0041–0.0116 against 0.0006–0.0085 for the prior); the prior is *better* on two of three pairs. This is the expected outcome, not a bug: average confidence is provably unbiased under calibration, and our production and calibration distributions match by construction, so there is no slice shift for stratification to exploit.

What provenance does buy is the quantity an operator actually acts on, which no unstratified estimator can express at all: accuracy conditioned on inference path. We recover a primary-path accuracy of 0.6605 as 0.6613 and a secondary-path accuracy of 0.4267 as 0.4699 on the object pair, and the analogous split on the other two. We also report a systems result that survives independently of the estimation question: the two-tier pipeline holds yield at 1.0 up to an injected primary-failure rate of 0.5, where a primary-only pipeline falls to 0.553, and the classical secondary is bit-for-bit reproducible across processes (0 mismatches in 5,000 records per pair). We argue the honest claim is the per-path estimate and the bounded-latency determinism, not a new estimator, and we name the baseline that beat us.

All numbers in this paper are produced by behavioural engine models on synthetic traces with fixed seeds. They are labelled SYNTHETIC throughout and none of them is a measurement of a production YOLO, FaceNet or PaddleOCR stack.

1 Introduction

Video-analytics platforms in the field do not run one model per task. They run a learned primary engine and, when it times out, crashes, or fails a health probe, a classical secondary: a Haar cascade behind a CNN face matcher, a template matcher behind a learned plate reader, a background-subtraction blob detector behind an object detector. The pipeline is designed so that a frame is never silently dropped. The consequence is a mixed output stream in which some records came from a strong model and some from a weak one, and the mix changes minute to minute with the health of the primary.

An operator staffing a control room asks a simple question about that stream: *how much of what I am looking at right now is right?* There are no labels. The obvious answer is to average the model's confidence over the batch. The tempting answer, once the pipeline already records which engine produced each record, is to stratify: estimate accuracy per inference path from a labelled calibration set, then reweight by today's observed path mix.

This paper reports that the tempting answer does not beat the obvious one, states why it cannot under the conditions we and most deployments operate in, and identifies the narrower claim that does survive.

1.1 The negative result, stated plainly

Let $\hat{a}_{\text{prior}}$ be the global calibration prior: the accuracy measured on a held-out labelled set, applied unchanged to production. Let $\hat{a}_{\text{prov}}$ be the provenance-stratified estimator: per-path accuracies from the same calibration set, reweighted by the production path mix. On three engine pairs with 3,000 calibration and 17,000 production records each, the absolute errors are 0.0041, 0.0116 and 0.0054 for $\hat{a}_{\text{prov}}$ against 0.0006, 0.0085 and 0.0061 for $\hat{a}_{\text{prior}}$. The prior wins on two of three, and the differences are inside the 95% interval on the true accuracy itself. Confidence-only estimation is worse than both (0.0105–0.0140) because our engines are imperfectly calibrated (ECE 0.0127–0.0167), which is a property of the engine models, not a defence of stratification.

Kivimäki et al. [11] give the reason. Average confidence is an unbiased estimator of accuracy whenever the scores are calibrated, with no distributional assumption beyond that. Stratification helps only when the calibration and production distributions differ *within* the strata used — the slice-shift setting Chen et al. [3] were built for. Our production stream is drawn from the same generator as our calibration set. There is no shift. There is therefore nothing for provenance to correct, and it would be surprising if it won.

1.2 Contributions

1. An operating-regime rule for the fallback decision, with a single break-even quantity a deployment can evaluate against its own cost of a missing record, measured across three engine pairs and five failure rates (§6).

2. A negative result: provenance stratification does not improve whole-batch label-free accuracy estimation in a matched-distribution two-tier pipeline, with the theoretical reason for why not (§3).

3. The narrower positive claim that does hold: per-path accuracy is recoverable from the same machinery and is not expressible by any unstratified estimator (§4), together with an honest report of where that recovery is loose (secondary-path error up to 0.043).

4. A systems result independent of the estimation question: the two-tier pipeline buys bounded latency and cross-process determinism, and pays for them in accuracy (§5). We report the loss rather than omit the baseline.

5. Guidance for practitioners, stated as conditions rather than as a recommendation to adopt anything (§9).

2 Setting

2.1 The deployed two-tier pipeline

Each analytic task is a pair (primary, secondary). We study three: object detection, face matching, and automatic number-plate recognition. The dispatcher calls the primary under a deadline. On timeout, exception, or a failed health probe it calls the secondary and marks the record. The secondary is always called; there is no drop path.

2.2 The record

Every emitted record carries nine fields, of which four matter here: the entity key, the engine confidence, the `source` field naming the engine that produced it, and the `path` field taking the value `primary` or `secondary`. Provenance is therefore free at estimation time — it is already written for audit reasons — which is exactly why the stratified estimator is worth testing and exactly why a negative result is worth publishing rather than quietly dropping.

2.3 Threat to validity we accept up front

The engines here are behavioural models parameterised by a difficulty scale, not the real stacks. The determinism result transfers to any pipeline whose secondary is a fixed-function classical algorithm. The accuracy numbers do not transfer and should not be read as characterising YOLO, FaceNet or PaddleOCR.

3 Why the obvious estimator wins

Write $Y \in \{0,1\}$ for record correctness, S for the model score, and $P \in \{\text{primary}, \text{secondary}\}$ for the path. Calibration means $\mathbb{E}[Y \mid S = s] = s$. Then

$$\mathbb{E}[\bar{S}] \;=\; \mathbb{E}\big[\mathbb{E}[Y \mid S]\big] \;=\; \Pr(Y = 1),$$

so average confidence is unbiased for batch accuracy without reference to P at all [11]. Conditioning on P can reduce variance, but the variance of a mean over 17,000 records is already far below the bias we are trying to detect: the 95% interval on true accuracy is $[0.6287, 0.6432]$ for the object pair, a width of 0.0145, which is wider than every estimator gap we measure.

Stratification pays only under shift. Let w_p be the calibration path share and w_p' the production share. The prior errs by $\sum_p (w_p' - w_p)\, a_p$ where a_p is per-path accuracy; the stratified estimator cancels exactly this term. Our generator holds $w_p' \approx w_p$, so the term it cancels is near zero and the estimator inherits only the extra estimation noise of two smaller calibration strata. That is the mechanism behind Table 1, and it is a property of the deployment, not of the idea. A deployment whose primary degrades diurnally — night-time face failures, monsoon plate failures — has real $w_p' \neq w_p$ and should expect the opposite result. We could not manufacture that shift honestly without also manufacturing the accuracy conclusion, so we report the matched case and say so.

4 What stratification does buy

Table 2 is the case for keeping provenance. The operational question is rarely "what is batch accuracy" and usually "is the degraded path carrying enough of tonight's output that I should

Pair	**True**	**Prov. strat.**	**Global prior**	**Conf. only**
objects	0.6352	0.6406 (0.0054)	**0.6413 (0.0061)**	0.6247 (0.0105)
faces	0.5949	0.5833 (0.0116)	**0.5863 (0.0085)**	0.5808 (0.0140)
anpr	0.5794	**0.5835 (0.0041)**	0.5800 (0.0006)	0.5676 (0.0118)

Table 1: **Whole-batch accuracy estimation (synthetic).** Absolute error in parentheses. $n_{\text{cal}} = 3{,}000$, $n_{\text{prod}} = 17{,}000$. The global prior is better than provenance stratification on two of three pairs and never worse by more than 0.003. **This table is the negative result.**

	Primary path		**Secondary path**		
Pair	True	Estimate	True	Estimate	**ECE**
objects	0.6605	0.6613	0.4267	0.4699	0.0154
faces	0.6328	0.6207	0.3647	0.3557	0.0127
anpr	0.6155	0.6248	0.4053	0.3844	0.0167

Table 2: **Per-path accuracy (synthetic).** The gap between paths is 0.21–0.27 and is recovered to within 0.001–0.043. No unstratified estimator produces these columns at all. The secondary-path estimate on the object pair is the loosest (0.043); we do not claim uniform tightness.

stop acting on it". The paths differ by 0.21 to 0.27 in accuracy. An unstratified estimator reports a single number that is a weighted average of two very different populations and is therefore correct about a quantity nobody needs.

We report the loose case rather than the mean of the four: the object-pair secondary estimate is off by 0.043, roughly 10% relative. The secondary stratum is the smaller one, and its calibration sample is correspondingly thin. A deployment that wants tight per-path numbers must size the calibration set per stratum, not per batch. That is a design consequence, and it is the useful sentence in this section.

5 The systems result, and the baseline that beats it

Three things are true at once in Table 3, and papers in this area usually report only the first.

Yield. The two-tier pipeline emits a record for every frame at every injected failure rate up to 0.5. A primary-only pipeline degrades linearly to 0.553. If the downstream consumer is a tracker that needs an observation per frame, this difference is the whole system.

Determinism. The classical secondary produced identical output across independent processes on all three pairs: 0 mismatches in 5,000 records each. This is not automatic. Our first implementation keyed off Python's builtin `hash()`, which is salted per process, so two runs of the same script disagreed while the unit tests passed. Stable hashing (`zlib.crc32`) fixed it. Anything that must reproduce across processes cannot hash strings with `hash()`; we record the trap because it is the kind of defect that survives a green test suite.

The loss. Retrying the primary beats the two-tier pipeline on accuracy-over-all at every failure rate we tested, by 0.014 to 0.032. This is not surprising — a successful retry returns a primary-quality record where the fallback returns a secondary-quality one — but it is decisive for how the contribution should be phrased. The fallback is defensible where a repeated call is *not* free and the failure is *not* transient: where the primary is down rather than flaky, where the deadline is hard, or where the retry would cost a GPU slot that another camera needs. Its wins are bounded latency (Table 4) and reproducibility, not accuracy. A paper that omits the retry column is making a claim its own data does not support.

Fail rate	Two-tier yield	Two-tier acc.	Primary-only yield	Primary-only acc.	Retry acc.
0.05	1.000	0.6353	0.9309	0.6073	**0.6496**
0.10	1.000	0.6354	0.8919	0.5892	**0.6508**
0.20	1.000	0.6274	0.8023	0.5349	**0.6450**
0.35	1.000	0.5897	0.6782	0.4491	**0.6214**
0.50	1.000	0.5698	0.5530	0.3715	**0.5934**

Table 3: **Yield and accuracy against injected primary-failure rate, object pair (synthetic, $n = 10{,}000$ per row).** Two-tier never drops a frame. It also never beats retry on accuracy-over-all. Both facts belong in the paper.

Pair	Mean (ms)	p99 (ms)	Secondary share	Accuracy
objects	41.7	47.2	0.108	0.6281
faces	55.4	58.1	0.144	0.5921
anpr	81.8	100.0	0.171	0.5830

Table 4: **Latency of the fallback path (synthetic).** The p99 is bounded by construction: a fallback is one extra call with a known cost, whereas a retry schedule has an unbounded tail.

6 The operating regime: when does each option win?

Section 5 establishes that retry returns better records and the two-tier pipeline returns more of them. Stated that way the comparison is a tie, and a paper that stops there has told the reader nothing they can act on. This section turns it into one number.

6.1 A break-even for the cost of a gap

Score the consumer's view of the stream. A returned-and-correct record is worth 1; a returned-and-wrong record is worth 0; a frame for which nothing is returned is worth $-m$, where m expresses how many correct records one gap costs that particular consumer. Writing a_2 for two-tier accuracy over all frames, a_r and y_r for retry's accuracy over all frames and its yield, and noting that the two-tier pipeline has yield 1 by construction:

$$U_{\text{two-tier}} = a_2, \qquad U_{\text{retry}} = a_r - m\,(1 - y_r).$$

These are equal at

$$m^\star \;=\; \frac{a_r - a_2}{1 - y_r}. \tag{1}$$

Retry wins when a gap costs less than $m^\star$; the two-tier pipeline wins when it costs more. Everything a deployment needs to supply is m, which is a statement about its own consumer and not about our engines.

6.2 Reading the table

Three things in Table 5 are worth stating plainly.

The regime boundary moves by two orders of magnitude. At a 5% primary failure rate $m^\star$ is 14.5 to 71.5: a gap would have to be worth dozens of correct records before the two-tier pipeline is justified, and almost no consumer values a gap that highly. At 50%, $m^\star$ is 0.11 to 0.45: a gap need only be worth a fraction of a correct record. The same two systems swap places, and the thing that moves them is the failure rate, not any property of the engines.

$m^\star$ **crosses** 1 **between failure rates of** 0.2 **and** 0.35**.** Below that band, a consumer would have to regard a gap as strictly worse than a wrong answer for the fallback to pay. Above it, treating a gap and a wrong answer as equally bad — the natural default when the downstream

Pair	**Fail rate**	a_2	a_r	y_r	a_r|**returned**	$m^\star$
objects	0.05	0.6353	0.6496	0.9998	0.6497	71.50
objects	0.10	0.6354	0.6508	0.9984	0.6518	9.63
objects	0.20	0.6274	0.6450	0.9898	0.6516	1.73
objects	0.35	0.5897	0.6214	0.9633	0.6451	0.86
objects	0.50	0.5698	0.5934	0.9028	0.6573	**0.24**
faces	0.05	0.6094	0.6165	0.9997	0.6167	23.67
faces	0.10	0.5940	0.6138	0.9975	0.6153	7.92
faces	0.20	0.5832	0.6237	0.9908	0.6295	4.40
faces	0.35	0.5526	0.6012	0.9605	0.6259	1.23
faces	0.50	0.5248	0.5695	0.9012	0.6319	**0.45**
anpr	0.05	0.6048	0.6077	0.9998	0.6078	14.50
anpr	0.10	0.5940	0.6040	0.9983	0.6050	5.88
anpr	0.20	0.5817	0.6072	0.9932	0.6114	3.75
anpr	0.35	0.5533	0.5819	0.9599	0.6062	0.71
anpr	0.50	0.5383	0.5499	0.8984	0.6121	**0.11**

Table 5: **Break-even gap cost (synthetic, n = 10,000 per row).** $m^\star$ from Eq. 1. Retry wins below it, the two-tier pipeline above it. The column a_r|returned is retry's accuracy on the frames where it returned anything, and it barely moves with the failure rate — retry's problem at high failure rates is not that its answers get worse, it is that it stops answering.

consumer is a tracker that needs an observation per frame — already favours the fallback. That band is the practical answer to the title question on our data.

Retry's conditional accuracy is flat. Across failure rates from 0.05 to 0.50, a_r|returned stays within 0.6078–0.6573 for objects and 0.6050–0.6121 for ANPR. Retry does not degrade; it abstains. That is why the comparison cannot be settled on accuracy alone and why $m^\star$, which prices the abstention, is the right summary.

6.3 What the break-even does not capture

Two costs sit outside Eq. 1 and push in the fallback's favour.

The first is the latency tail. A fallback is one additional call with a known cost, so its p99 is bounded by construction: 47.2, 58.1 and 100.0 ms on the three pairs (Table 4). A retry schedule of up to three attempts has a tail set by the number of attempts and the backoff, and we did not instrument it. Where a deadline is hard, the bounded tail can decide the question before accuracy is considered at all.

The second is what a repeated call costs. Eq. 1 implicitly assumes a retry is free. On a platform where the primary engine holds a scarce decode slot or a GPU session, a retry consumes capacity that another camera needs, and the true comparison is against the work displaced rather than against zero. We do not model that here; a deployment that is resource-bound should.

There is also a failure mode retry cannot address at all. Our failures are injected independently per frame, so a retry is always a fresh draw and usually succeeds. When the primary is *down* rather than flaky — a crashed process, an unloadable model, an expired licence — every attempt fails, retry's yield collapses to zero, and the fallback is the only option that returns anything. Our generator cannot express that case, and it is the one where the fallback's value is least disputable.

7 Baselines

We compare against average confidence [11], the global calibration prior, difference-of-confidences [8], and the average threshold confidence family [6], both global and per-stratum. Slice-based estimation [3] is the closest prior work and is the correct comparison for the stratified estimator: it formalises exactly the reweighting we perform, with provenance as the slicing function. Our result is that under matched distributions this reduces to the prior, which is consistent with what Chen et al. [3] claim rather than in tension with it.

Per-stratum ATC ties with provenance stratification on the per-path metric to within 0.01 on all three pairs, which we state here rather than in an appendix. The honest reading is that *recording the path* is what buys the per-path number; the particular estimator layered on top of it is not the contribution.

8 Related work

The two-tier architecture is anticipated. NoScope [10] cascades a cheap specialised model in front of an expensive one with a difference detector and a learned threshold. Neural Simplex [14] switches from a learned controller to a verified fallback under a safety monitor. BranchyNet [16] exits early on confident samples. IDK cascades [17] formalise the abstain-and-defer step, and the selective-prediction line [4, 7] gives its decision theory. FrugalGPT [2] is the same pattern under a budget. We add nothing architectural to this literature and say so.

Label-free accuracy estimation is likewise a developed field: ATC [6], agreement-based estimation [1], AutoEval [5], DoC [8], projection norm [18], confidence optimal transport [12], and the older estimation-without-labels line [15]. Calibration quality is the binding constraint on all of them [9, 13], which is why our confidence-only column is the weakest.

Where we sit is therefore narrow: we test a specific combination (provenance-as-slice on a two-tier video pipeline) that the literature makes plausible, find it does not pay under matched distributions, and report the conditions under which it would.

9 Implications for system design

We state these as conditions rather than as a recommendation to adopt anything, because the paper's finding is that both pieces of machinery we examined are justified only in part of the range.

On the fallback. Measure the primary's failure rate and price a gap before building a secondary. If the failure rate sits below roughly 0.2 and a gap is worth less than a correct record, retry is the better system on our data and the secondary is unbuilt work. Build the secondary when the failure rate is high, when the failure is persistent rather than transient, when the deadline is hard enough that an unbounded retry tail is unacceptable, or when a repeated call consumes a resource another camera needs. Those four conditions, not a general claim of superiority, are what the fallback rests on.

On the estimator. Record the inference path — it is nearly free and is usually required for audit anyway — and then report accuracy per path. Do not expect the batch number to improve, and do not build a stratified batch estimator on the strength of a matched-distribution evaluation, because under matched distributions there is nothing for it to correct (§3). Revisit that conclusion only with evidence of genuine path-mix shift between calibration and production, which is the condition under which the slice-based literature predicts a gain.

On calibration. Confidence-only estimation was the weakest method on all three pairs (0.0105–0.0140 absolute error against 0.0006–0.0085 for the prior), and its error tracks our engines' calibration error (ECE 0.0127–0.0167). Since average confidence is unbiased exactly when scores are calibrated, effort spent on calibration buys estimation accuracy directly, and is likely a better investment than a more elaborate estimator.

On sizing the calibration set. Per-path estimates are only as good as the smaller stratum. Our loosest per-path figure is the object pair's secondary estimate, off by 0.043, and the secondary stratum is the thin one. Size the calibration set per stratum rather than per batch.

10 Limitations

1. The engines are behavioural models with a difficulty parameter. Accuracy figures characterise the generator.
2. Failure is injected at a fixed rate, independently per frame. Real primary failures are bursty and correlated with the conditions that also make frames hard, which would create the very shift that would make stratification win. The same assumption flatters retry throughout §6: independent failures mean each attempt is a fresh draw, so correlated or persistent failure moves $m^\star$ down and the regime boundary towards the fallback. We report the independent case because it is what the generator produces, and we expect it is the case least favourable to our own pipeline.
3. $m^\star$ prices a gap but not a retry. Where a repeated call consumes a scarce decode slot, the correct comparison includes the displaced work, and Eq. 1 understates the fallback's case.
4. The determinism claim is about the secondary engine's output given a frame. It is not a claim that the two-tier system as a whole is deterministic; which path a frame takes depends on wall-clock timeouts.
5. Single estate, single generator, three pairs. Nothing here is validated against operational footage.

11 Venue

We consider this a workshop paper and say so rather than leaving it to a reviewer. Neither the two-tier architecture (§8) nor the estimator is new, and the paper's value is a pair of operating rules for practitioners plus a negative result reported with its baseline intact. Systems-and-machine-learning and ML-safety workshop tracks are the natural fit. A main-track submission would be asking a novelty-seeking committee to reward a paper whose finding is that two pieces of machinery were not worth building, and we would expect it to be rejected for the reason the paper itself gives.

12 Conclusion

Recording the inference path of every record is cheap, is already required for audit, and does not improve label-free estimation of whole-batch accuracy. Under calibration and matched distributions the unstratified prior is unbiased and the stratified estimator has nothing left to correct; our measurements agree with the theory to within the confidence interval on the target quantity. What the path field does buy is the per-path accuracy split, which no unstratified estimator can produce and which is the number an operator acts on. Separately, the two-tier pipeline buys yield and cross-process determinism at a measured cost in accuracy-over-all relative to retry. We recommend that deployments record provenance, report accuracy per path, and stop expecting the batch number to improve.

On the fallback the answer is a boundary rather than a verdict. Retry returns better records and the two-tier pipeline returns more of them, and which matters depends on one quantity a deployment can measure for itself: what a gap in the stream costs. On our three engine pairs that break-even runs from 71.5 correct records per gap at a 5% primary failure rate down to 0.11 at 50%, crossing unity between failure rates of 0.2 and 0.35. Below that band a secondary engine is unbuilt work; above it, it is the system. Both of this paper's findings have the same shape, and it is the shape we would like the reader to take away: the simple option wins over most of the range, and the useful contribution is knowing where the range ends.

Reproduction

```
cd code
pip install -r requirements.txt
python -m pytest -q
python -m prresearch.p2_fallback.experiment
```

Fixed seeds; `results/p2_fallback.json` is byte-identical across runs and across processes. Code and data: `https://github.com/amitduabits/PRAHARI-P2-NegativeResult`

References

[1] Christina Baek, Yiding Jiang, Aditi Raghunathan, and J. Zico Kolter. Agreement-on-the-line: Predicting the performance of neural networks under distribution shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL `https://proceedings.neurips.cc/paper_files/paper/2022/hash/7a8d388b7a17df480856dff1cc079b08-Abstract-Conference.html`.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[3] Mayee Chen, Karan Goel, Nimit S. Sohoni, Fait Poms, Kayvon Fatahalian, and Christopher Ré. Mandoline: Model evaluation under distribution shift. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, 2021. URL `https://proceedings.mlr.press/v139/chen21i.html`.

[4] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.

[5] Weijian Deng and Liang Zheng. Are labels always necessary for classifier accuracy evaluation? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. URL `https://openaccess.thecvf.com/content/CVPR2021/html/Deng_Are_Labels_Always_Necessary_for_Classifier_Accuracy_Evaluation_CVPR_2021_paper.html`.

[6] Saurabh Garg, Sivaraman Balakrishnan, Zachary C. Lipton, Behnam Neyshabur, and Hanie Sedghi. Leveraging unlabeled data to predict out-of-distribution performance. In *International Conference on Learning Representations (ICLR)*, 2022.

[7] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL `https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html`.

[8] Devin Guillory, Vaishaal Shankar, Sayna Ebrahimi, Trevor Darrell, and Ludwig Schmidt. Predicting with confidence on unseen distributions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. URL https://openaccess.thecvf.com/content/ICCV2021/html/Guillory_Predicting_With_Confidence_on_Unseen_Distributions_ICCV_2021_paper.html.

[9] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

[10] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. Noscope: Optimizing neural network queries over video at scale. *Proceedings of the VLDB Endowment (PVLDB)*, 10(11):1586–1597, 2017. doi: 10.14778/3137628.3137664.

[11] Juhani Kivimäki, Jukka K. Nurminen, Jakub Białek, and Wojtek Kuberski. Confidence-based estimators for predictive performance in model monitoring. *Journal of Artificial Intelligence Research (JAIR)*, 82:209–240, 2025. doi: 10.1613/jair.1.16709.

[12] Yuzhe Lu, Yilong Qin, Runtian Zhai, Andrew Shen, Ketong Chen, Zhenlin Wang, Soheil Kolouri, Simon Stepputtis, Joseph P. Campbell, and Katia Sycara. Characterizing out-of-distribution error via optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/38fd51cf36f28566230a93a5fbeaabbf-Abstract-Conference.html.

[13] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua V. Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html.

[14] Dung T. Phan, Nicola Paoletti, Xin Zhang, Radu Grosu, Scott A. Smolka, and Scott D. Stoller. Neural simplex architecture. In *NASA Formal Methods (NFM)*, volume 12229 of *Lecture Notes in Computer Science*, pages 97–114, 2020. doi: 10.1007/978-3-030-55754-6_6.

[15] Emmanouil Antonios Platanios, Avrim Blum, and Tom M. Mitchell. Estimating accuracy from unlabeled data. In *Proceedings of the 30th Conference on Uncertainty in Artificial Intelligence (UAI)*, 2014. URL https://auai.org/uai2014/proceedings/individuals/313.pdf.

[16] Sergey Teerapittayanon, Bradley McDanel, and H. T. Kung. Branchynet: Fast inference via early exiting from deep neural networks. In *Proceedings of the 23rd International Conference on Pattern Recognition (ICPR)*, 2016. URL https://ieeexplore.ieee.org/document/7900006/.

[17] Xin Wang, Yujia Luo, Daniel Crankshaw, Alexey Tumanov, Fisher Yu, and Joseph E. Gonzalez. IDK cascades: Fast deep learning by learning not to overthink. In *Proceedings of the 34th Conference on Uncertainty in Artificial Intelligence (UAI)*, 2018. URL https://www.auai.org/uai2018/proceedings/papers/212.pdf.

[18] Yaodong Yu, Zitong Yang, Alexander Wei, Yi Ma, and Jacob Steinhardt. Predicting out-of-distribution error with the projection norm. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *PMLR*, 2022. URL https://proceedings.mlr.press/v162/yu22i.html.